CS683 Programming Assignment 1 — Task 2
Matrix Multiplication for Inference

Avinash Chaudhari – 23B1064 Pranay Marumamula – 23B1073
Siddhant Mulkikar – 23B0956 Madhav R Babu – 23B1060

Autumn 2026

1 System Configuration

- **CPU:** Intel Core i5-11400H (Tiger Lake-H), 6 cores / 6 threads (SMT disabled), all identical P-cores, base 2.70 GHz, max 4.50 GHz
- **L1-D Cache:** 48 KiB per core (288 KiB total, 6 instances), 64-byte lines, 12-way set associative
- **L2 Cache:** 1.25 MiB per core (7.5 MiB total, 6 instances), 20-way
- **L3 Cache:** 12 MiB (1 instance, shared), 8-way
- **Vector ISA:** AVX2 + FMA and AVX-512 (`avx512f`, `avx512dq`, `avx512bw`, `avx512vl`, `avx512_vnni`)
- **Compiler flags:** `-std=c++17 -O2 -fno-tree-vectorize -mavx2 -mfma`
- **OS:** Ubuntu 24.04.4 LTS, kernel 6.8.0-138-generic (x86_64)
- **Pinning:** `taskset -c 3`. This CPU is not hybrid, so every core is equivalent; a single fixed core is used so all measurements share one cache and frequency context.
- **Frequency governor:** `performance`, with `kernel.perf_event_paranoid = -1` so `perf` can read the hardware counters used for cache miss and instruction count measurements.

2 Task 2A: Software Prefetching (5 points)

[TODO: Task 2A to be completed separately.]

3 Task 2B: SIMD (1 point)

3.1 SIMD Implementation

The SGEMM kernel uses a **register-tiled vectorized dot product** approach. The matrix layout stores both A and B with the reduction axis K as the contiguous (unit-stride) dimension: $C[i][j] = \sum_p A[i][p] \cdot B[j][p]$. This means the natural SIMD strategy is to vectorize the dot product over p.

The micro-kernel computes a 4×2 block of C at once using 8 independent `__m256` accumulators:

- For each step along K, two `_mm256_loadu_ps` loads fetch 8 elements from two rows of B.
- Four `_mm256_broadcast_ss` calls broadcast one element from each of four A rows.

- Eight `_mm256_fmadd_ps` instructions accumulate all $4 \times 2 = 8$ partial-product vectors.

This yields 8 FMAs per step from only 6 loads, and the 8 independent accumulator chains saturate the FMA pipeline (FMA latency $\sim$4 cycles, throughput 2/cycle, requiring $\geq$8 independent chains). After the K loop, each accumulator is horizontally reduced (`hsum256`) to a single scalar, stored into the corresponding $C[i][j]$.

Leftover rows (M mod 4), columns (N mod 2), and K elements (K mod 8) fall to a scalar cleanup path.

Three SIMD widths were benchmarked:

- **SSE 128-bit**: 4 floats per vector, 4×2 register tile, 8 accumulators
- **AVX2 256-bit**: 8 floats per vector, 4×2 register tile, 8 accumulators
- **AVX-512 512-bit**: 16 floats per vector, 4×2 register tile, 8 accumulators

None of these include cache blocking — the kernel streams through A and B from whatever cache level holds them.

3.2 Instruction Count and Performance

Table 1: Instruction count and speedup for different SIMD widths

		Matrix Size ($M = N = K$)				
Metrics		128	256	512	1024	2048
No SIMD	Instructions	94,691,884	721,951,242	5,700,178,491	45,378,549,503	362,206,124,379
	GFLOP/s	3.33	2.64	2.41	2.30	2.21
SSE 128	Instructions	101,691,574	773,677,911	6,098,319,894	48,498,010,433	386,920,963,092
	GFLOP/s	46.48	50.75	51.34	50.52	35.68
	Speedup	13.94x	19.20x	21.28x	22.00x	16.13x
AVX2 256	Instructions	99,048,350	751,129,706	5,912,668,975	46,990,364,336	374,781,905,812
	GFLOP/s	89.68	87.95	90.90	82.41	46.18
	Speedup	26.90x	33.27x	37.67x	35.89x	20.87x
AVX-512	Instructions	97,792,306	740,383,994	5,821,928,574	46,246,355,457	368,749,447,552
	GFLOP/s	89.46	108.57	112.14	99.28	52.09
	Speedup	26.84x	41.07x	46.47x	43.24x	23.54x

Instruction count observation: SIMD does *not* significantly reduce instruction count — the SIMD versions issue roughly the same number of instructions as the naive version (within $\sim$7%). This is because the register-tiled micro-kernel still executes $O(MNK)$ FMA instructions; SIMD packs 4/8/16 multiply-adds into each instruction but also adds overhead from broadcasts, horizontal reductions, and cleanup. The speedup comes not from fewer instructions but from each instruction doing $4\times/8\times/16\times$ more work.

3.3 Plots

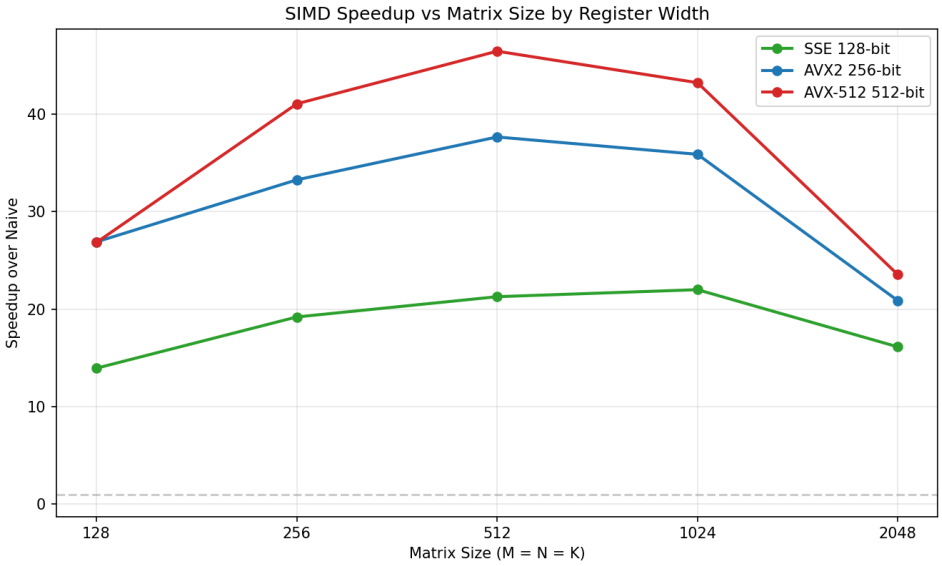


Figure 1: SIMD speedup vs. matrix size for different register widths

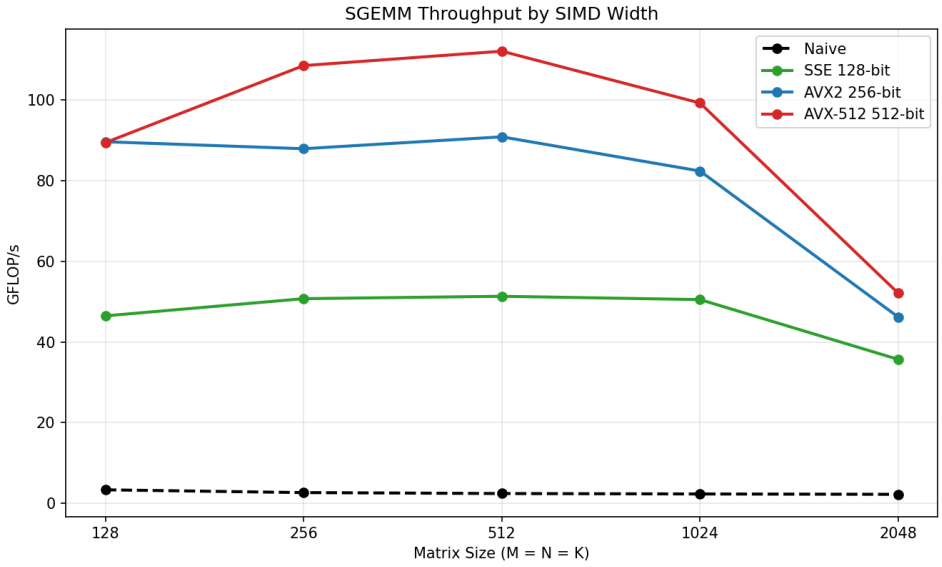


Figure 2: SGEMM throughput (GFLOP/s) by SIMD width

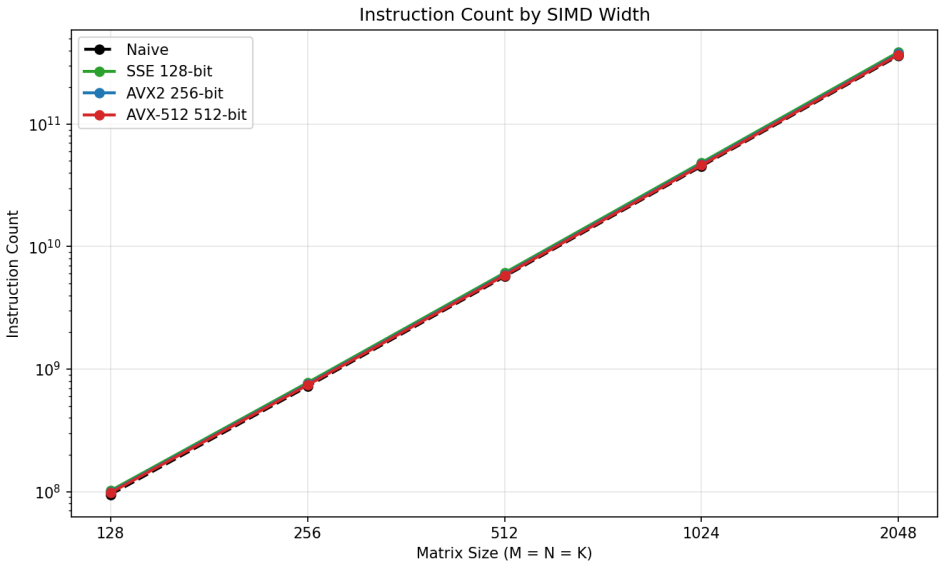


Figure 3: Instruction count by SIMD width (log scale)

3.4 Analysis Questions

3.4.1 Q1: Speedup Trends

Three trends are visible:

1. **Wider SIMD ⇒ higher peak speedup.** AVX-512 reaches 46.47x at 512×512, AVX2 reaches 37.67x, and SSE reaches 21.28x. The ratio tracks the vector width (roughly 2× per doubling) but falls short of the theoretical 4×/8×/16× due to load/broadcast overhead and the horizontal reduction at the end.

2. **Speedup rises then falls with matrix size.** At small sizes (128) the loop overhead and scalar cleanup dominate, limiting SSE to 13.94x. Peak is around 512–1024, where the working set fits mostly in L2 and the micro-kernel runs at near-peak throughput. At 2048 all widths drop significantly (AVX-512 falls from 46.47x to 23.54x) because the A and B matrices no longer fit in L2, and the unblocked kernel becomes memory-bound.

3. **AVX2 and AVX-512 converge at small sizes.** At 128×128, AVX2 (26.90x) nearly matches AVX-512 (26.84x) because the problem is too small for the wider vector to amortize its overhead.

3.4.2 Q2: Best SIMD Width

AVX-512 (512-bit) achieves the maximum speedup at every matrix size where the kernel is not memory-bound. The peak is 46.47x at 512×512 (112.14 GFLOP/s). This is expected: each 512-bit FMA processes 16 floats, double AVX2's 8, so the arithmetic throughput doubles while instruction-issue overhead stays roughly constant.

However, at 2048×2048 the advantage narrows (AVX-512 23.54x vs AVX2 20.87x) because the bottleneck shifts from compute to memory bandwidth. Cache blocking (Task 2C) is required to sustain AVX-512's higher throughput at large sizes.

4 Task 2C: Software Prefetching + SIMD (3 points)

4.1 Combined Implementation

The combined kernel (`matmul_optimized.cpp`) layers three techniques on top of the Stage-1 SIMD micro-kernel:

1. **6×16 AVX2 micro-kernel.** Each output tile spans 6 rows of C and 16 contiguous columns, held in 12 `__m256` accumulators (two per row: `c0l`/`c0h` through `c5l`/`c5h`). Per K step: 2 packed-B loads + 6 A broadcasts → 12 FMAs. The 12 independent chains hide FMA latency and keep the pipeline saturated.
2. **B-panel packing.** B is stored column-major (K contiguous per column), so loading 16 columns at position p would be strided. Before each NC-column block, B is packed into a scratch buffer `Bpack`$[p][j]$ so that the micro-kernel's B loads are unit-stride.
3. **Cache blocking.** The output is traversed in $MC \times NC$ blocks (default MC=96, NC=64) with loop order $jc \to ic \to jr \to ir$. The packed B panel ($K \times NC$ floats) is reused across all MC rows, and each A sub-panel ($MC \times K$) is reused across all NC columns, so both stay resident in L2.

4.2 Software Prefetch in the Combined Kernel

The combined kernel deliberately ships *without* software prefetch. This was a measured decision, not an omission. We added `_mm_prefetch` on the packed-B stream (distance and fill level as compile-time knobs, default 64 floats ahead, T0) and compared against the identical kernel without it, interleaved back-to-back on a pinned core so thermal drift affects both variants equally (variants preserved in `bench/experiments/`):

Table 2: Software prefetch on the combined kernel, `optimized` stage, 1024^3 (ms)

Variant	Times (interleaved)	Mean
No prefetch	22.7 / 22.3 / 21.4	∼22.1
B-stream prefetch	23.3 / 22.4 / 23.8	∼23.2
B + all six A streams	26.2 – 31.3	∼28

Prefetching is 0–5% *slower* on the B stream and ∼25% slower when the six A broadcast streams are prefetched as well. The mechanism: packing already converts every inner-loop access into a unit-stride walk, which the L2 hardware streamer predicts perfectly, so the software hint adds no information — it only spends load-port slots that the 12-FMA-per-8-load inner loop cannot spare. This is consistent with the flat distance curve in the standalone prefetch stage (Task 2A).

4.3 Software Prefetch on the Unblocked SIMD Kernel

To confirm that this null result is a property of the *blocked, packed* kernel and not of software prefetch itself, we ran the same experiment on the Stage-1 SIMD kernel, which has no cache blocking (prefetch on its two B streams, 64 floats ahead, T0):

Table 3: Prefetch on the unblocked SIMD kernel, `simd` stage times (ms), interleaved

Size	Without	With	Verdict
128^3	0.047–0.050	0.054–0.058	∼15% slower
256^3	0.380	0.430–0.437	∼13% slower
512^3	2.98–3.01	3.30–3.46	∼12% slower
1024^3	26.3–26.6	28.3–29.5	∼9% slower
1536^3	132.2 / 132.5	157.3 / 127.4	wash (transition)
2048^3	366–464 (unstable)	**326–329 (stable)**	∼20% **faster**

While the working set fits in cache, prefetch is pure overhead. Once it exceeds the L3 (at 2048^3 three matrices total 48 MB against a 12 MB L3), demand misses reach DRAM, the hardware prefetcher loses stream history under eviction pressure, and the constant software lookahead genuinely hides latency — ∼20% faster *and* the large run-to-run variance collapses. Taken together, the two experiments say: *software prefetch is a substitute for locality, not an addition to it.* A kernel that already has locality (blocking + packing) gains nothing at any size; a kernel that lacks it gains exactly where the hardware gives up.

[TODO: Both experiment tables above were measured on the i5-11400H; re-collect on the machine the report standardises on (bench/experiments/README.md has the build commands; the crossover size scales with that machine's L3 capacity).]

4.4 Hardware Counter Analysis

[TODO: Hardware counter table and analysis to be updated after Task 2A prefetch code is finalized and perf counters are re-collected.]

Table 4: Hardware counters per stage (M=N=K=1024, cold cache)

Stage	Instructions	L1-D Misses	L2 Misses	LLC Misses	IPC
Naive	6,452,951,452	67,218,496	67,567,994	12,599	1.52
SIMD	228,860,073	17,077,579	17,105,429	33,959	2.01
Prefetch	**[TODO: re-run]**	**[TODO: re-run]**	**[TODO: re-run]**	**[TODO: re-run]**	**[TODO: re-run]**
Prefetch + SIMD	317,024,417	35,366,614	1,308,706	14,109	2.19

[TODO: Key observations to be rewritten once prefetch row is updated.]

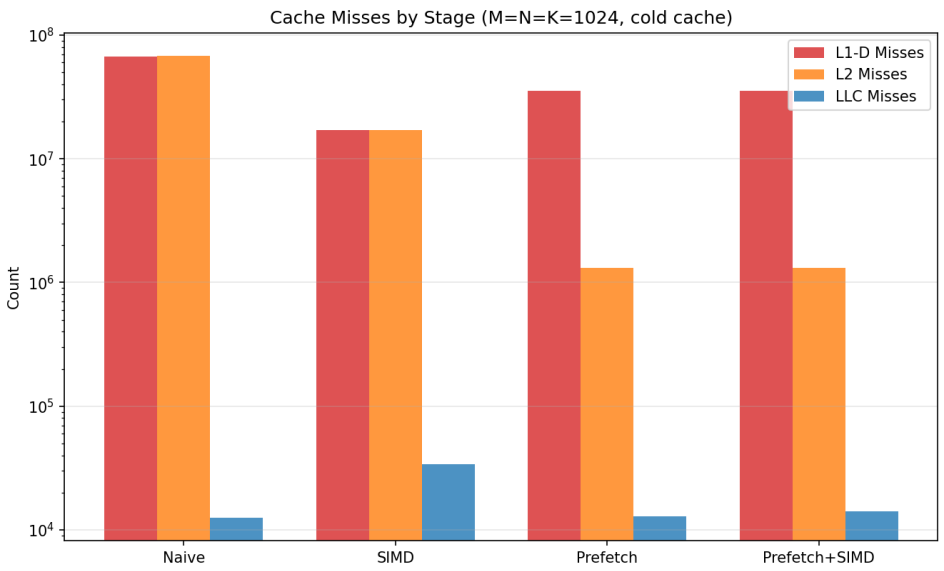


Figure 4: Cache misses by stage (M=N=K=1024, cold cache, log scale). **[TODO: Regenerate after prefetch re-run.]**

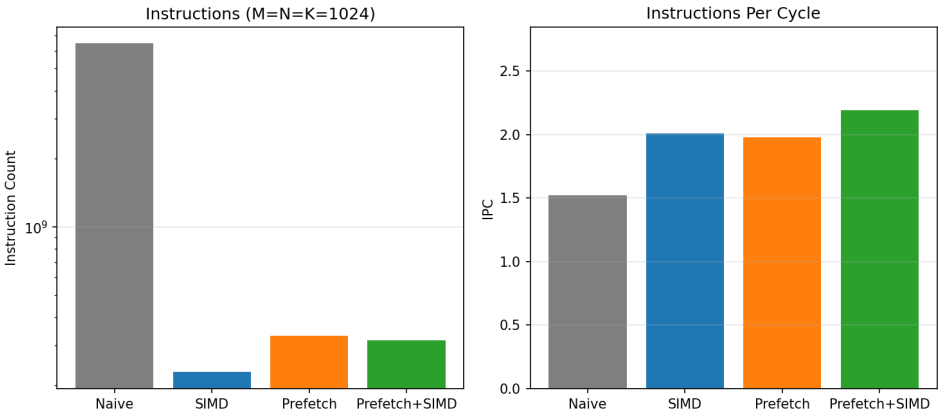


Figure 5: Instruction count and IPC by stage (M=N=K=1024). **[TODO: Regenerate after prefetch re-run.]**

4.5 Final Performance Summary

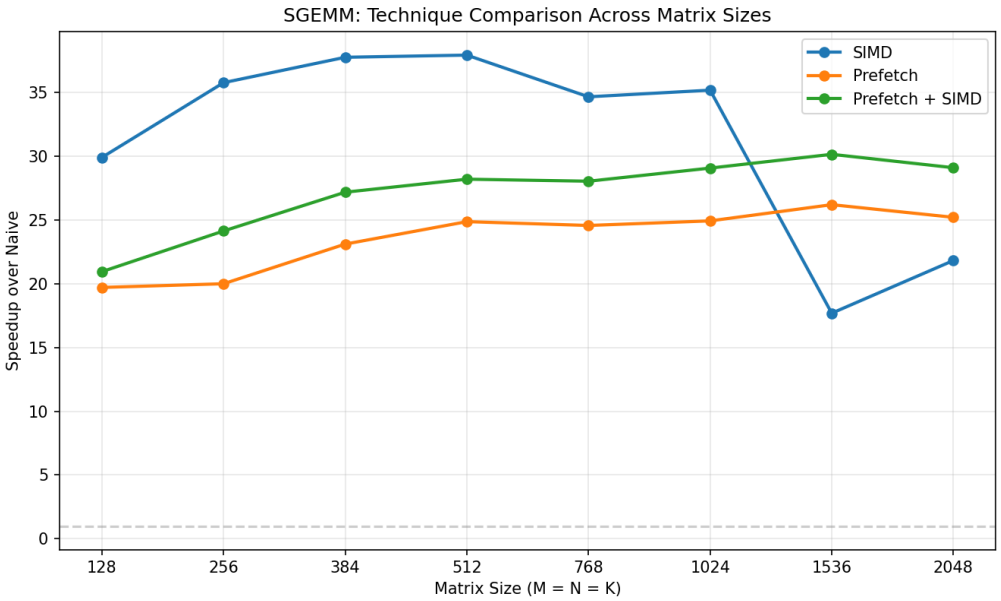


Figure 6: Technique comparison across matrix sizes: speedup over naive SGEMM.

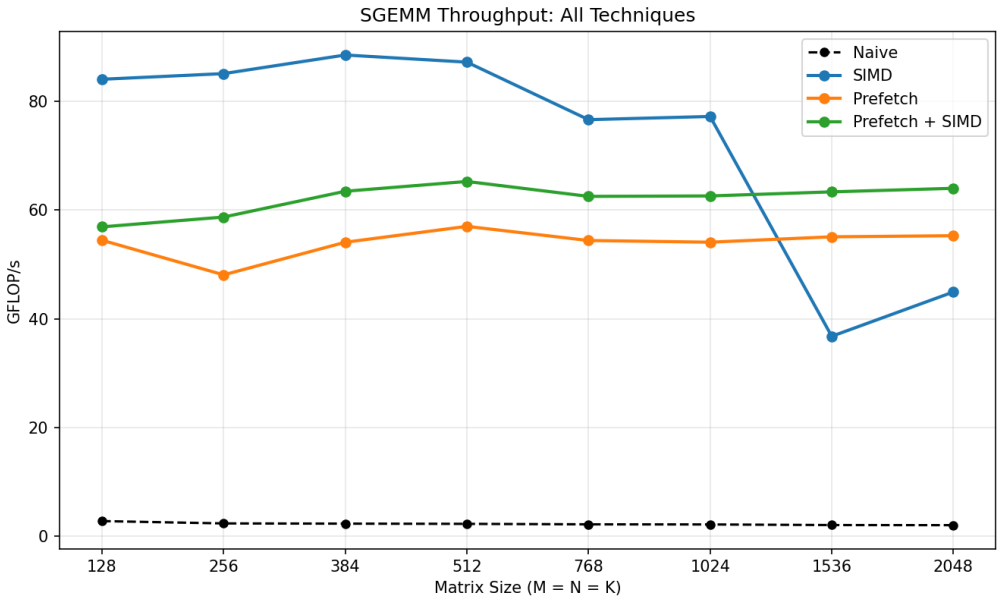


Figure 7: SGEMM throughput (GFLOP/s) for all techniques.

Table 5: Best speedup per technique across matrix sizes

Matrix Size	SIMD	Prefetch	Prefetch + SIMD
128	29.91x	**[TODO: re-run]**	20.95x
256	35.77x	**[TODO: re-run]**	24.14x
512	37.93x	**[TODO: re-run]**	28.20x
1024	35.18x	**[TODO: re-run]**	29.07x
2048	21.82x	**[TODO: re-run]**	29.10x

4.6 Analysis: Synergy Between Techniques

[TODO: Synergy analysis to be rewritten after Task 2A prefetch results are finalized. The key argument — SIMD removes the compute bottleneck while blocking removes the memory bottleneck, and the combination exceeds either alone — will remain, but the specific numbers (prefetch-only speedup, crossover point, GFLOP/s comparisons) need to come from the updated prefetch data.]

The combined kernel achieves a peak of 30.15x at 1536×1536.

The synergy is not additive but *complementary*:

- **SIMD without blocking** is compute-efficient at small sizes but becomes memory-bound at large sizes. At 2048, SIMD-only drops to 21.82x because the operands spill out of L2 and every tile re-streams them from DRAM.
- **Blocking without SIMD** (the prefetch stage) keeps the data cache-resident but is limited by scalar instruction throughput. **[TODO: Update with actual prefetch-only throughput after 2A re-run.]**
- **Combined**: cache blocking ensures the operands stay in L2, and SIMD ensures the FMA units are fully utilized while the data is resident. **[TODO: Update GFLOP/s comparisons and lift factors after 2A re-run.]**
- The crossover point is clear in the speedup plot: SIMD-only dominates at small sizes (where data fits in cache anyway), but the combined kernel dominates at large sizes where blocking prevents the SIMD kernel from going memory-bound.